

Adversarial Attacks (AA)

Robust Deep Learning

Lara Braukmann (457008) and Alina Shcherbakova (463616)

RWTH Aachen University, Germany
{lara.braukmann, alina.shcherbakova}@rwth-aachen.de

1 Introduction

Deep learning has achieved remarkable success across a wide range of tasks, particularly in computer vision, natural language processing, and speech recognition. However, despite their impressive performance, it was discovered that even state-of-the-art neural networks in image classification are highly vulnerable to adversarial examples – original images that are slightly modified in a way that is not noticeable to humans but causes the neural network to misclassify them as something else [CW17].

The robustness of machine learning models refers to their ability to maintain correct predictions *even* in the presence of such adversarial examples [Mad+18]. Creating such inputs is known as an adversarial attack and is used both by attackers to exploit models and by researchers to evaluate model robustness. This vulnerability of neural networks is especially relevant in safety-critical applications, such as autonomous driving or biometric authentication, where adversarial attacks can lead to drastic consequences.

Most adversarial attacks have so far focused on the image domain, where small and often imperceptible changes of pixel values cause neural networks to misclassify the input [Sze+14; GSS15].

However, adversarial vulnerabilities are not limited to visual data. In [CW16], audio data has been perturbed, where hidden voice commands have successfully manipulated voice assistants such as Alexa.

Furthermore, adversarial attacks also pose a risk in text classification, malware detection and spam recognition [BR18]. In these cases, the structure of the input data is different than for visual and audio data, but the vulnerability of machine learning models to the created perturbations remains the same [PY20].

The discovery of adversarial examples revealed a fundamental weakness in deep learning models: their decision boundaries are not consistent with human perception. Perturbed inputs that are mostly imperceptible to humans remain valid model inputs, while leading to incorrect model predictions. This shifted the focus of model robustness research from improving generalisation on natural data to optimising the performance of the model against adversarial attacks [Mad+18; Sch+18].

In this seminar report, we focus on adversarial attacks in the image domain, as adversarial examples were first introduced in *this* context by Szegedy et al. [Sze+14], which sparked a surge of research interest and led to the development of numerous attack and defence methods within the visual domain.

Our goal is to explain the development from foundational concepts to state-of-the-art attacks. To achieve this, we outline how adversarial attack methods have progressed and where they can be applied.

1.1 Overview of Basic Concepts

In order to ensure clarity, we briefly introduce key neural network concepts.

Neural networks are machine learning models inspired by the structure and function of the human brain. They consist of interconnected nodes, called neurons, organized into layers. Each connection between neurons has an adjustable weight, and training involves tuning these weights to minimise prediction errors, measured using a loss function.

Typically, neural networks receive input vectors whose form depends on the specific domain. For instance, in an image classification task with 28x28 pixel grayscale images (such as the MNIST dataset [Den12]), the input vector $\mathbf{x}$ is represented as a flattened vector of pixel intensity values, with each entry ranging between 0 (black) and 1 (white), resulting in a 784-dimensional vector [Sze+14]:

$$\mathbf{x} = (x_1, x_2, \dots, x_{784}), \quad 0 \leq x_i \leq 1.$$

The neural network processes this input through multiple layers, ultimately producing an output vector of length K , where K represents the number of possible classes. This output vector is denoted as $\mathbf{z} = (z_1, z_2, \dots, z_K)$.

The entries z_k in this output vector are called **logits** [Goo24]. Logits themselves are raw scores and do not directly represent probabilities. However, the predicted class can still be determined directly from logits by choosing the class with the highest logit value, i.e., $\arg \max_k z_k$. Typically, logits are transformed into probabilities using the *softmax* function [GBC16]:

$$\text{softmax}(z_k) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}.$$

This normalisation yields a probability distribution over the K classes, ensuring that the outputs sum to 1 and each value lies between 0 and 1. Without normalisation, logits can vary significantly in magnitude, complicating comparisons between different samples.

Neural networks learn by adjusting their weights so that the predicted probabilities best match the true labels of the inputs.

Loss function. The discrepancy between a prediction and the true label y is measured by a loss function $\mathcal{L}(\theta, \mathbf{x}, y)$, where θ are the current model parameters (such as the weights), $\mathbf{x}$ the input sample, and y the true label. In optimisation terms, such a loss function is called the **objective function** – the function the learning algorithm seeks to minimise during training [GBC16]. For multi-class classification, the common choice is the **cross-entropy loss** [Ter+25].

Let $\hat{\mathbf{y}} = (\hat{y}_1, \dots, \hat{y}_K)$ denote the predicted probability distribution obtained from the softmax layer and the true label y is represented as a one-hot encoded vector $y = (y_1, y_2, \dots, y_K)$, where $y_k = 1$ if the correct class is k , and 0 otherwise. Over n samples, the average cross-entropy loss is

$$\mathcal{L}(\theta, \mathbf{x}, y) = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_k^{(i)} \log(\hat{y}_k^{(i)}) = -\frac{1}{n} \sum_{i=1}^n \log(\hat{y}_{k_i}^{(i)}),$$

because in each sample only the term for the correct class k_i ($y_{k_i}^{(i)} = 1$) contributes.

Gradient Descent. To minimise the loss, parameters θ are updated iteratively via *gradient descent* [Rud16]:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta, \mathbf{x}, y),$$

where $\eta > 0$ is the learning rate, and $\nabla_{\theta} \mathcal{L}$ represents the gradient of the loss with respect to the parameters. Common variants include stochastic gradient descent and adaptive optimisation methods such as Adam [Rud16].

Backpropagation. Gradients required for gradient descent are computed using backpropagation. After a *forward pass* computes the network output and the loss, a single *backward pass* – **backpropagation** – computes all gradients needed for gradient descent. Backpropagation identifies how each weight impacts the overall loss, enabling targeted updates that iteratively reduce the prediction error.

2 Adversarial Examples

The concept of **adversarial examples** was first introduced by Szegedy et al. [Sze+14], who defined them as inputs that have been modified by adding imperceptible, non-random *perturbations* – small changes, such that a deep neural network’s correct prediction changes to an incorrect one even

though the input appears unchanged to the human eye. These perturbations are generated by optimising the input to maximise the model’s prediction error.

Formally, for an original input x , a similarity metric $\mathcal{D}$, and a small threshold $\varepsilon > 0$, an input x' is considered a valid adversarial example if

$$\mathcal{D}(x, x') \leq \varepsilon \quad \text{and} \quad f(x') \neq y,$$

where f denotes the classifier and y is the true label of x .

Notably, Szegedy et al. also observed that adversarial examples often transfer across models; that is, they tend to remain misclassified by other neural networks, even when those networks are trained with different hyperparameters (number of layers, regularisation or initial weights) or on different training data. This suggests a certain degree of robustness in the adversarial examples themselves and indicates that their effect is not simply due to the variability that naturally arises from different runs of backpropagation-based training.

In Figure 1, an example of such adversarial images is presented. The figure illustrates that even such small perturbations - although imperceptible to the human eye - can cause a deep neural network like AlexNet [KSH12] to misclassify the input.

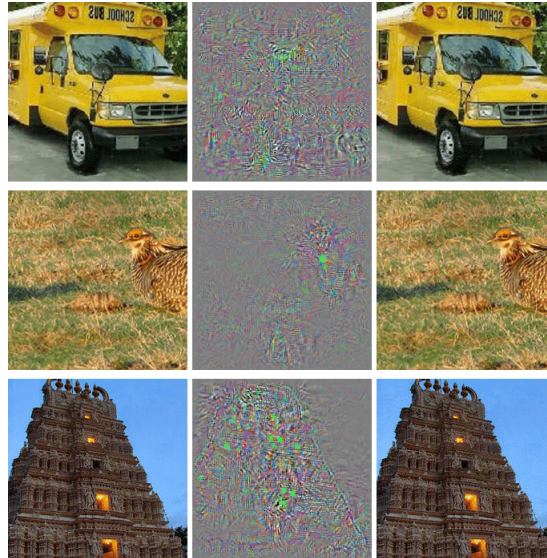


Figure 1: Adversarial examples generated for AlexNet, as presented by Szegedy et al. [Sze+14]. The left column shows correctly classified inputs. The center column displays perturbations (magnified $\times 10$), and the right column shows adversarial examples that appear visually unchanged but are misclassified as “ostrich, *Struthio camelus*.”

1

Zhang et al.[ZL20] extended the discussion by cataloguing three salient characteristics of adversarial examples:

1. **Transferability.** Adversarial examples are often transferable across models – adversarial examples crafted for a fully known (white-box) model often still mislead unseen (black-box) models that solve the same task even when the target architecture or training data differ - thereby enabling attacks on otherwise unknown models. (The terms “white-box” and “black-box” models are explained in more detail later.)

¹ High-resolution images were originally available via the authors, but the link is now inactive. A version was retrieved from ResearchGate: https://www.researchgate.net/publication/259440613_Intriguing_properties_of_neural_networks

2. **Regularisation effect.** Incorporating adversarial examples during training can act as a form of data-dependent regularisation, often improving robustness or revealing weaknesses of the model, though at the cost of increased computational overhead what makes lighter methods such as dropout or weight-decay more attractive.
3. **Adversarial instability.** Success in the digital domain does not guarantee robustness in the physical world: natural transformations such as viewing angles, lighting changes, or minor translations can neutralise carefully designed perturbations, allowing the input to be correctly classified again and forcing attackers to craft perturbations that remain effective under such real-world transformations.

2.1 Hypotheses on the Existence of Adversarial Examples

In their original work, Szegedy et al. [Sze+14] did not offer a definitive explanation for the existence of adversarial examples, but instead highlighted it as an open question. As part of their discussion, they proposed a possible explanation: adversarial examples might belong to a set of inputs with extremely low probability under the data distribution, making them rarely observed in standard test sets. However, this set could be dense in the input space – similar to how the rational numbers are dense in the real numbers – implying that adversarial examples may occur near virtually every test instance. The authors emphasized that a deeper understanding of the frequency and structure of such examples was lacking and encouraged further research on this topic.

Goodfellow et al. [GSS15] later challenged speculative hypotheses that attributed adversarial examples to the extreme nonlinearity of deep networks combined with insufficient model averaging and weak regularisation. They argued that such assumptions are unnecessary: the inherent linear behaviour of high-dimensional models is sufficient to explain their vulnerability. Even simple linear models exhibit adversarial examples when the input dimensionality is high enough. Although deep networks make use of nonlinear activation functions, their overall behaviour in practice – when using units such as ReLUs, maxout, or sigmoid functions – remains surprisingly linear. This is often due to architectural choices or tuning strategies that encourage models to operate in relatively linear regimes. From this perspective, adversarial vulnerability emerges not as a flaw of overfitting, but as an inherent property of how modern neural networks represent and process high-dimensional input data.

2.2 Applications of Adversarial Examples

Adversarial examples can primarily be used in two contrasting ways: as tools for attacking machine learning models, and as a means of assessing and improving model robustness.

In offensive settings, adversarial examples serve for **adversarial attacks**, where deliberately crafted inputs are used to manipulate a model's predictions. Their transferability property makes them particularly interesting in black-box scenarios, where the attacker does not have access to the internal parameters of the target model. This makes adversarial examples a practical threat to deployed systems, including those in sensitive domains like autonomous driving and biometric verification.

Conversely, adversarial examples can also be used prior to deployment to evaluate a model's robustness by intentionally attacking it with adversarial examples, thereby revealing the model's weaknesses. A prominent defence strategy is **adversarial training** [Mad+18], in which each training batch is augmented with adversarial variants of the input data. By optimising the model on these harder examples, the network could learn to resist small, malicious perturbations and thus becomes more robust. This improvement in resilience, however, often comes at the cost of increased computational requirements.

Thus, adversarial examples are not only a source of vulnerability but also a valuable instrument for stress-testing and hardening neural network models.

3 Adversarial Attacks

Adversarial attacks are the intentional creation of inputs that mislead machine learning models. While adversarial examples characterise these inputs, adversarial attacks describe the concrete methods employed to create them.

Adversarial attacks refer to the intentional creation of specific inputs – adversarial examples – that are designed to mislead machine learning models. While adversarial examples are the manipulated inputs that cause misclassification, adversarial attacks describe the methods used to generate them.

These attacks are typically divided into [BR18]:

1. **White-box attacks**, which include full access to model parameters and gradients.
2. **Grey-box attacks** with partial access to the target model, for example to architecture, some internal parameters or substitute models. These exploit the transferability of adversarial examples, as discussed above, where inputs created to mislead one model can also fool other models with different architectures and training data.
3. **Black-box attacks**, which only cover access to input-output behaviour, no knowledge of internal information.

The appropriate type of an adversarial attack is typically defined by its use cases. To evaluate model robustness and mitigate weaknesses, white-box attacks can be used as the required access to the model is usually provided. [CW17; GSS14] However, for malicious purposes, black-box attacks will often be applied because there is often no internal knowledge about the target model. [PMG17; BRB18a]

The attack goals of an adversarial attack can be classified as **targeted** or **untargeted**. While targeted attacks aim to push input to a specific incorrect label, untargeted attacks seek any misclassification. In Large Language Models, the creation of targeted adversarial inputs is harder than untargeted ones, mainly due to the optimisation complexity [Gei+24].

Additionally, a targeted attack can also aim to manipulate the input of a model without being noticeable, while changing the semantic interpretation.

An example of such an attack is presented in [Els+18], where Elsayed et al. demonstrate that adversarial examples can fool both machine learning models and humans. In their study, a human control group was exposed to the manipulated inputs under time-limited conditions and was unable to recognise the perturbations. This shows that the attack remains unnoticeable even in real-world scenarios, where human oversight is involved. Therefore, these attacks are particularly challenging to identify and defend against.

The adversarial attacks that are discussed in this paper are limited to **digital input**. However, in [Eyk+18] Eykholt et al. introduced the concept of **physical** adversarial attacks, for instance placing a sticker on top of a stop sign to fool deep learning models employed in traffic recognition systems. This highlights the risks of adversarial vulnerabilities that are no longer contained to the digital world, especially in safety-critical applications.

3.1 Threat Norms

Threat Norms define the mathematical constraints on the perturbations used in adversarial attacks, by specifying the magnitude and structure of the perturbation applied to the input. These constraints ensure that the perturbation remains within a predefined region. Additionally, they reflect the practical capabilities of the adversary, such as the perceptibility of changes to humans or the technical practicality of modifying specific input features.

In adversarial machine learning, the perturbation region is defined as an ℓ_p -ball with a fixed radius ϵ [GSS14; Mad+18]:

$$\mathcal{B}_p(x, \epsilon) = \{x' \in \mathbb{R}^n : \|x' - x\|_p \leq \epsilon\}$$

A *ball* in this context refers to the set of all points (perturbed inputs) that have a distance of at most ϵ from the original input x , where the distance is measured by the ℓ_p -norm. The ℓ_p -norm quantifies the magnitude of the perturbation δ as follows:

$$\|\delta\|_p = \left(\sum_{i=1}^n |\delta_i|^p \right)^{1/p}$$

where n is the dimensionality of the input and δ_i is the perturbation applied to the i -th feature. The most common choices for ℓ_p are:

- ℓ_0 : constrains the *number of features* that can be changed. For instance, limiting the attack to modify only a few pixels in an image.
- ℓ_2 : constrains the *Euclidean distance* of the perturbation, which distributes the modifications across multiple features.
- ℓ_∞ : constrains the *maximum change per feature*, which ensures that no single pixel is altered by more than ϵ .

The choice of the norm influences the visual or semantic detectability of adversarial examples. For instance, ℓ_∞ attacks constrain the maximal individual change of an input feature and are designed to be imperceptible in every input dimension, while ℓ_0 attacks create localised perturbations that are generally more noticeable, but only affect a few input features [Mad+18; GSS14].

4 Adversarial Attack Methods

L-BFGS Attack. The first successful adversarial attack was introduced by Szegedy et al. [Sze+14] as a method for generating adversarial examples – a concept that was also formally defined for the first time in the same paper. They formulated the task as a box-constrained optimisation problem and solved it using the L-BFGS algorithm. Although foundational, the attack is highly time-consuming [GSS15; Yua+17] and has largely been superseded by more efficient gradient-based methods such as FGSM and PGD, which are discussed in more detail later in this seminar report.

4.1 Fast Gradient Sign Method (FGSM)

The **Fast Gradient Sign Method (FGSM)** is an untargeted, single-step white-box attack, originally introduced by Goodfellow, Shlens, and Szegedy [GSS14] to generate ℓ_∞ -bounded perturbations. FGSM is considered one of the most influential adversarial attacks, as it marked the beginning of systematic research in the field and inspired many subsequent developments [Akh+21].

Given a differentiable classifier f_θ , an input $x \in [0, 1]^d$ with true label y , and a loss function $\mathcal{L}(\theta, x, y)$, FGSM first constructs the *perturbation vector*

$$\eta = \epsilon \operatorname{sign}(\nabla_x \mathcal{L}(\theta, x, y)),$$

where

- $\nabla_x \mathcal{L}(\theta, x, y)$ is the gradient of the loss with respect to the input x ;
- $\operatorname{sign}(\cdot)$ is applied element-wise, yielding a vector of $+1$ or -1 entries;
- $\epsilon > 0$ limits the maximum change to any single feature (pixel).

Because every component of η equals $+\epsilon$, $-\epsilon$, or 0 the attack satisfies the ℓ_∞ bound $\|\eta\|_\infty = \epsilon$. The adversarial example is obtained with a single update:

$$x^{\text{adv}} = x + \eta.$$

This step moves the input in the direction that most rapidly increases the loss, yet keeps each pixel perturbation within $\pm\epsilon$. The attacker must know the gradient, so FGSM assumes a white-box setting, but it requires only one backward pass, making it computationally cheap and a popular baseline for both research and *adversarial training*.

FGSM is not guaranteed to find adversarial examples close to the original input; it was designed to be fast rather than optimal. A single FGSM step may fail when *gradients are masked* or the loss landscape is locally flat.

4.2 Basic Iterative Method (BIM)

Kurakin, Goodfellow, and Bengio [KGB16] proposed the extension of FGSM - **Basic Iterative Method (BIM)**, also known as *Iterative Gradient Sign* [CW17].

Starting from the original input $x^{(0)} = x$, BIM performs T small FGSM steps of size α and projects the result back into the ℓ_∞ ball of radius ε after each step:

$$x^{(t+1)} = \text{clip}_{x,\varepsilon}(x^{(t)} + \alpha \text{sign}(\nabla_{x^{(t)}} \mathcal{L}(\theta, x^{(t)}, y))), \quad t = 0, \dots, T-1.$$

Here $\text{clip}_{x,\varepsilon}(\cdot)$ enforces the constraint $\|x^{(t+1)} - x\|_\infty \leq \varepsilon$ and keeps pixel values within the valid pixel range. The final adversarial example is $x^{\text{adv}} = x^{(T)}$.

Using $\text{sign}(\nabla_x \mathcal{L})$ yields an *untargeted* BIM attack; replacing it with $-\text{sign}(\nabla_x \mathcal{L}(\theta, x, t))$ moves the sample toward a chosen target class t .

BIM attains significantly higher success rates than one-step FGSM, at the expense of T forward-backward passes. An important contribution of this attack is that the authors demonstrated its applicability in the physical world, thereby inspiring further research in the physical domain [Akh+21]. This experiment was one of the first to demonstrate that adversarial examples can remain effective outside the purely digital domain [KGB16]. They printed adversarially perturbed ImageNet images [Den+09] on paper, took photographs of them with a mobile-phone camera under various angles and lighting conditions, and showed that these images still misled the classifier – whereas the printed clean versions were correctly classified.

4.3 Projected Gradient Descent

Projected Gradient Descent (PGD) is an iterative adversarial attack method, first introduced by Aleksander Madry and Aleksandar Makelov in [Mad+18]. In their work, the authors compared PGD to FGSM and CW attacks, demonstrating that PGD achieves higher attack success rates in the majority of cases.

The goal of their paper is to formalise adversarial robustness as a worst-case optimisation problem. To solve this problem, PGD is used to create strong adversarial examples. This is done by approximating the inner maximisation part of the saddle point problem:

$$\max_{\delta \in \mathcal{S}} \mathcal{L}(\theta, x + \delta, y)$$

First, a clean input x with its correct label y is taken from the dataset and a small random noise within the allowed bounds is added. The bounds define how much the input x is allowed to change in order to create the adversarial example.

Then, the following steps are repeated a fixed number of times to approximate the maximum:

1. The gradient of the loss for a fixed model θ is calculated.
2. A small step into the direction that increases the loss is made.
3. The result is projected back into the allowed perturbation region $\mathcal{S}$. This leads to the hard detectability of adversarial examples created with PGD, as the allowance ensures that the perturbation δ is small enough that the adversarial example is almost identical to x .

The greatest improvement of PGD was its ability to generalise among attack methods and datasets, as existing attacks were often only effective in specific cases. [Mad+18]

PGD was the first **universal first-order adversarial technique** that could achieve strong adversarial examples, while only using gradient information, which is called first-order information. This led to the ability of PGD to generalise.

In [Mad+18], Madry and Makelov specifically demonstrated that:

1. If a model is able to defend itself against PGD attacks, it has a significantly higher chance of being able to defend itself against a wide range of other attacks, including FGSM and BIM.
2. Adversarial training with PGD is effective across different datasets, including MNIST, CIFAR-10 [KNH+14] and ImageNet.

3. Models trained to defend against PGD have also been shown to defend better against black-box attacks from independently trained networks.

Conceptually, PGD is very similar to BIM: it performs multiple small gradient steps and enforces an ℓ_p -constraint after each update. Unlike BIM, however, PGD typically starts from a random point within the ϵ -ball and is often run with multiple restarts. These design choices make PGD markedly stronger and less prone to overestimating robustness in the presence of gradient masking than one-step FGSM or BIM.

While PGD was long considered the standard benchmark for evaluating adversarial robustness – and one of the most influential attacks due to its ability to reliably generate strong adversarial examples [Akh+21] – more recent evaluations often rely on ensemble-based methods like AutoAttack, which includes two new extended versions of PGD [CH20], due to their stronger theoretical guarantees.

4.4 Carlini-Wagner (CW) Attacks

Carlini and Wagner [CW17] proposed in 2016 three closely related targeted, white-box adversarial attacks that seek the smallest perturbation needed to force a model to classify an input as a chosen target class t . Each variant minimises one of the following distance metric – ℓ_0 , ℓ_2 , ℓ_∞ .

The authors stated that these norms were chosen because earlier work considered them reasonable approximations of human perceptual similarity. Later studies, however, show that a small ℓ_p distance is neither necessary nor sufficient for imperceptibility [SBR18]. Nonetheless, Mangal et al. [Man+23] argue that bounding perturbations in the ℓ_p norm should be seen as a minimal safety requirement when deploying models in security-sensitive settings. Moreover, as those authors note (see citations therein), several studies (2019,2020) report that improving ℓ_p robustness can also enhance a model’s resistance to non- ℓ_p perturbations.

Carlini and Wagner [CW17] report that their CW- ℓ_2 and CW- ℓ_0 attacks achieve 100 % success across MNIST, CIFAR-10, and ImageNet, while requiring significantly lower distortion than the prior state-of-the-art methods (e.g. iterative FGSM, BIM) famously breaking the then state-of-the-art defence, **defensive distillation** [Pap+16]. Although the method assumes gradient access, many CW examples still transfer to black-box models.

The CW- ℓ_∞ variant matches the quality of earlier ℓ_∞ -based attacks, but achieves a higher success rate, and can even succeed on ImageNet dataset by modifying as little as a single pixel [CW17].

Core optimisation problem. For an input $x \in [0, 1]^d$ with true label y and target label $t \neq y$, CW solves

$$\min_{\delta} \|\delta\|_p + c f(x + \delta) \quad \text{s.t.} \quad x + \delta \in [0, 1]^d,$$

where $c \geq 0$ balances distortion and attack success. The surrogate loss

$$f(x') = \left(\max_{i \neq t} Z_i(x') - Z_t(x') \right)^+, \quad (u)^+ := \max(u, -\kappa) \quad (1)$$

uses logits $Z_i(\cdot)$ and a confidence margin κ : $f(x') \leq 0$ only when class t scores at least κ more than every rival class. Larger values of κ typically yield stronger transferability. However, for the purpose of simply achieving targeted misclassification, Carlini & Wagner set $\kappa = 0$, and only experimented with larger values to generate more transferable adversarial examples.

Remark. The following text omits a detailed discussion of the tuning procedure for the balancing constant c . In brief – c is typically selected via modified binary search.

CW- ℓ_2 Attack. The CW- ℓ_2 attack seeks to construct an adversarial example with the smallest possible ℓ_2 perturbation that causes the classifier to predict a specific target label t . This is achieved through the following steps:

1. *Re-parameterisation*: Introduce an *unconstrained optimisation variable* $w \in \mathbb{R}^d$ – that is, w is free to take any real value and is **not** subject to the pixel box-constraint. Map it to a valid image via

$$x(w) = \frac{1}{2}(\tanh w + 1) \in [0, 1]^d.$$

Because the $\tanh$ mapping automatically clips every coordinate to the range $[0, 1]$, we can optimise w in free Euclidean space while guaranteeing that the corresponding image $x(w)$ always satisfies the box-constraint. The initial point is

$$w^{(0)} = \operatorname{arctanh}(2x_{\text{orig}} - 1),$$

so that $x(w^{(0)}) = x_{\text{orig}}$.

2. Minimise the composite objective

$$\min_w \|x(w) - x_{\text{orig}}\|_2^2 + c f(x(w)),$$

where $f(\cdot)$ is the CW surrogate loss from Eq. (1). Optimisation is performed with **Adam** for a fixed number of iterations.

CW- ℓ_∞ Attack. Instead of evaluating robustness at a *fixed* budget ε , the CW- ℓ_∞ attack *adapts* its threshold: it iteratively searches for – and ultimately reports – the smallest infinity-norm perturbation that forces the input into the chosen target class. Hence, ε is not preset; the algorithm discovers the minimal $\|\delta\|_\infty$ on its own, and that value is returned as the attack’s distortion.

Key idea. The ℓ_∞ norm is *non-differentiable*, so CW replaces it with a **smooth** surrogate:

$$g_\tau(\delta) = \sum_{i=1}^d \max(\delta_i - \tau, 0),$$

which upper-bounds $\|\delta\|_\infty$ and approaches it as $\tau \downarrow 0$.

Optimisation loop.

1. Initialise $\tau = 1$ and set $w^{(0)}$ via the $\tanh$ re-parameterisation (as in CW- ℓ_2).
2. Solve in each iteration with Adam

$$\min_w g_\tau(x(w) - x_{\text{orig}}) + c f(x(w))$$

3. Let $\delta = x(w) - x_{\text{orig}}$ be the perturbation obtained at the end of step 2. If its infinity-norm satisfies $\|\delta\|_\infty < \tau$, update $\tau \leftarrow 0.9\tau$ and repeat step 2 to search for a smaller distortion. Otherwise terminate.

CW- ℓ_0 Attack. The attack aims to find an adversarial example while altering as few pixels as possible. In each iteration, the algorithm uses an ℓ_2 -based optimisation to obtain a perturbation, identifies pixels with the smallest effect on the classifier’s output, fixes those pixels to their original values, and repeats until only the minimal set of influential pixels remains to be modified.

Because the ℓ_0 norm is non-differentiable, Carlini and Wagner [CW17] solve the problem in two stages.

Two-stage procedure.

1. **Identify influential pixels (dense search).**

Optimise an objective that blends an ℓ_2 distortion term with an ℓ_1 sparsity surrogate:

$$\min_w \|x(w) - x_{\text{orig}}\|_2^2 + \lambda \sum_i |\delta_i| + c f(x(w)),$$

where $x(w) = \frac{1}{2}(\tanh w + 1)$ ensures the pixel box-constraint and $\delta = x(w) - x_{\text{orig}}$. The ℓ_1 term ($\lambda > 0$) pushes the optimiser to concentrate large magnitudes on the *most influential* pixels, yielding a dense yet informative perturbation.

2. Greedy pruning (sparse refinement).

Starting from that dense δ , iteratively:

- 2.1. set the pixel with the smallest absolute change to zero,
- 2.2. re-optimize stage 1 (with that pixel frozen at 0),
- 2.3. stop when the classifier no longer outputs the target label t .

The perturbation from the last successful round is the *sparsest* adversarial example found.

By breaking defensive distillation with 100% success across all three of their attacks, Carlini & Wagner demonstrated that many defences had been evaluated against attacks that were simply too weak, establishing a new state-of-the-art white-box attack [Che+17]. Their work underscored the need for rigorous benchmarks based on powerful attacks to evaluate model robustness. While the original Carlini-Wagner attacks remain an essential baseline, recent evaluations typically use AutoAttack [CH20].

4.5 AutoAttack

Until now, all adversarial attacks presented have used gradient-based methods to calculate adversarial examples. However, these attacks often fail to reliably evaluate the robustness of a model, mainly because of two key problems:

First, *gradient masking*, where models produce misleading gradients that make gradient-based attacks ineffective, which can lead to a false sense of security.

Second, many attacks rely on carefully *tuned hyperparameters*, such as the step size or number of iterations. If these parameters are not optimally chosen during evaluation, the attack may appear weaker than it actually is.

As a result, many defences that initially seemed promising were later broken by stronger or better-tuned attacks. This makes it difficult to assess the true robustness of models.

To combat these issues, in [CH20], Croce et al. propose a combination of four diverse parameter-free attacks, named **AutoAttack**, which provide a more reliable robustness evaluation.

Two of the four attacks in AutoAttack are based on **Auto-PGD (APGD)**. APGD is an improved white-box attack that builds on PGD and reduces some of its limitations.

While PGD uses a fixed *step size* which determines the change of the input, APGD starts with a large step size and automatically reduces it by half when the progress stagnates. This is done by progress monitoring with checkpoints. Specifically, the iterations are divided into checkpoints at which the algorithm evaluates whether the loss has been sufficiently reduced. If not, the step size is reduced by half and the optimisation restarts from the best checkpoint that has been *discovered* [CH20].

Additionally, PGD uses a fixed size of iterations which do not influence the execution of the optimisation algorithm. APGD divides the total number of iterations (N_{iter}) into an *exploration phase* and an *exploitation phase*. In the exploration phase, the algorithm uses a large step size to quickly explore the *range of possible adversarial examples*, while in the exploitation phase, the step size is progressively decreased to optimise the adversarial example. This means that the timing of the step size reductions and checkpoint placements depends on N_{iter} [CH20].

As a result of these changes, APGD becomes a *parameter-free* attack, where the only required parameter is the number of iterations N_{iter} .

To form the AutoAttack, these four attack methods are combined:

APGD with Cross-Entropy Loss. APGD with Cross-Entropy Loss (APGD-CE) applies Auto-PGD using *cross-entropy loss*. Cross-entropy loss measures how well the model predicts the correct class. For a model with logits $z \in \mathbb{R}^K$ and correct class y , the cross-entropy loss is given by:

$$\text{CE}(x, y) = -z_y + \log \left(\sum_{j=1}^K e^{z_j} \right)$$

In the context of adversarial attacks, the cross-entropy loss is maximised to decrease the certainty of the model in the correct class, thereby causing misclassification. As a result of that, APGD-CE is classified as an *untargeted attack*.

APGD-CE is particularly effective against randomised defences, such as adding random noise to inputs [CH20] because its adaptive step size allows it to overcome randomness that confuses fixed-step attacks.

However, Croce et al. identify a key limitation: when the logits of the model are scaled down, the differences between the logits decrease, which causes the softmax output to approach a uniform distribution. In this case, the cross-entropy loss hardly changes when the input changes, so the gradients become very small. As a result, gradient-based attacks like PGD or APGD struggle to find effective update directions, which makes it difficult to generate adversarial examples.

APGD with DLR Loss. To address this limitation of APGD-CE, Croce et al. propose APGD with DLR Loss (APGD-DLR). This attack uses a *targeted* loss function defined as:

$$\text{DLR}_{\text{targeted}}(x, y, t) = -\frac{z_y - z_t}{z_1 - \frac{z_3 + z_4}{2}},$$

where y is the correct class, t is the target class and z_1, z_3 and z_4 represent the largest, third-largest and fourth-largest logits, respectively. This loss is *shift- and scale-invariant*, which means that it is not affected by logit transformations that could otherwise cause gradient masking. APGD-DLR modifies the attack process by targeting specific classes. This approach forces the adversarial example to cross the decision boundary towards a predefined class, rather than any incorrect class. As a result, the attack becomes more controlled and produces a more accurate evaluation of the robustness of the model. Experiments in [CH20] show that APGD-DLR performs especially well against defences that try to mask gradients. For different models and datasets, APGD-DLR consistently achieves lower robust accuracy, which additionally demonstrates its strength in evaluating model robustness. [CH20].

Fast Adaptive Boundary Attack. Fast Adaptive Boundary Attack (FAB) is a white-box attack that finds adversarial examples with minimal change by iteratively projecting the input onto the decision boundary between the correct class and a target class. In AutoAttack, FAB is used in its *targeted* version, which searches for the smallest adjustment that changes the prediction of the model from the correct class y to a specific target class t . The targeted FAB optimisation solves:

$$\text{FAB}_{\text{targeted}}(x, y, t) = \arg \min_{x'} \|x' - x\|_p \quad \text{such that } f_t(x') \geq f_y(x'),$$

where x is the clean input, y is the correct class, t is the target class and $f_i(x')$ refers to the logit of class i for the changed input x' . The operator *arg min* returns the input x' that achieves the smallest perturbation on or beyond the allowed decision boundary.

FAB complements the gradient-based APGD attacks: While APGD focuses on maximising the loss, FAB directly targets the decision boundaries. Experiments in [CH20] show that FAB is especially effective on robust models, where gradient-based attacks alone often struggle to find small adversarial changes. The main limitation of FAB is its computational cost because its iterative boundary search requires more steps than simpler gradient-based attacks.

Square Attack. Square Attack is a *score-based* black-box attack that creates norm-bounded adversarial examples by applying random square-shaped modifications to the input. Unlike gradient-based attacks, Square Attack does not rely on gradient information or gradient approximations. Instead, to find adversarial examples, it does a random search over square regions of the input and only applies changes that increase the loss. Random search works well here because it does not depend on potentially unreliable gradient approximation and can still explore the input space effectively.

In [And+20], Andriushchenko et al. show that Square Attack outperforms other black-box attacks by requiring fewer model queries and achieving a higher success rate. Square Attack is even competitive with some white-box attacks, despite not using gradients [CH20].

In AutoAttack, Square Attack complements the white-box attacks to ensure that the combination of the four attacks can detect vulnerabilities even when gradient-based methods fail due to gradient masking or other defence mechanisms. The main downside of Square Attack is its need for a large number of iterations ($N_{\text{iter}} = 5000$ in AutoAttack) to achieve strong results, which makes it less query-efficient than white-box attacks.

There are two main improvements:

First, it removes the need for hyperparameter tuning by using parameter-free attacks. Second, it combines four diverse attacks that complement each other and uncover different model vulnerabilities [CH20]. These improvements make robustness evaluations more objective, as they no longer depend on the attack setup or the tuning choices of the user.

This consistent and automated evaluation approach led to the establishment of AutoAttack as a new benchmark for adversarial robustness. AutoAttack is the default evaluation method in *RobustBench*, a standardised platform for comparing the robustness of machine learning models [Lor+22]. RobustBench applies AutoAttack to over 120 models from recent top machine learning and computer vision papers. In almost all cases, AutoAttack finds lower robust accuracies than originally reported, which reveals weaknesses in many proposed defences.

However, despite its widespread adoption, AutoAttack has also been criticised. Lorenz et al. argue in [Lor+22] that in certain settings of AutoAttack, for instance a large perturbation size, the adversarial examples created are easy to detect. They also point out that results on low-resolution datasets like CIFAR-10 do not necessarily generalise to higher-resolution datasets. This limits the applicability of the benchmark in some real-world scenarios.

4.6 Substitute models

As noted earlier, adversarial examples often *transfer*: an input that fools one model is likely to mislead others, even when their architectures or training data differ. This property underpins **substitute-model attacks**. The attacker first trains a local *surrogate network*, crafts adversarial examples on it, and then sends those same inputs to the remote, gradient-hidden system.

Papernot et al. [Pap+17] were the first to demonstrate that such transfer-based attacks can reliably break real-world, commercial black-box services, highlighting their practical relevance and again confirming the transferability property of adversarial examples. *On Evaluating Adversarial Robustness* further observes that transfer attacks are especially effective against defences that rely on gradient masking [Car+19].

1. Seed collection. Draw a small public seed set $\mathcal{S}_0$ (e.g. 150 instances for MNIST); obtain labels from f_{target} to create the initial training set $\mathcal{S}_0$.

2. Surrogate training. Train substitute DNN $f_{\text{sub}}^{(k)}$ on $\mathcal{S}_p$.

3. Jacobian-based data augmentation. For every $x \in \mathcal{S}_p$ compute the Jacobian $J(x) = \nabla_x f_{\text{sub}}^{(k)}$ and synthesise $x' = x + \lambda \text{sign}(J(x))$, which lies close to the surrogate’s decision boundary. Collect the set $\mathcal{S}_{p+1}$ of all such x' .

4. Label & loop. Query f_{target} on $\mathcal{S}_{p+1}$, append the answers to obtain $\mathcal{S}_{p+1} = \mathcal{S}_p \cup \{(x', y')\}$, and repeat steps 2-4 for $K = 6-8$ rounds. The labelled set roughly doubles each round, yielding $2^K \times |\mathcal{S}_0|$ queries. The number of queries can be reduced with **reservoir sampling** without significant reduction of the substitute accuracy.

After the final surrogate $f_{\text{sub}}^{(K)}$ is trained, the attacker applies a standard white-box method (e.g FGSM) to craft adversarial examples x^{adv} , which are then sent to f_{target} . The surrogate typically achieves a **lower accuracy** than the real model, but that is acceptable: its sole purpose is to approximate the target’s decision boundary well enough for the crafted perturbations to transfer.

Papernot et al. stress that the *substitute* only needs to be *differentiable*, because its gradients drive both the Jacobian-based data augmentation and the subsequent white-box attack. In practice the surrogate does not have to be a deep network: the authors remark that simple, differentiable models such as *logistic regression* can serve the same role. Crucially, the target classifier may be non-differentiable: adversarial examples generated on the surrogate transferred successfully to

decision trees, SVMs, k -nearest neighbours, and logistic regression, with misclassification rates exceeding 80 % in most cases. This shows that the substitute model strategy generalises far beyond deep neural networks.

The Jacobian-augmentation loop used in the original approach is query-expensive – the number of queries grows exponentially with the number of iterations; more recent active learning strategies achieve similar results with one to two orders of magnitude fewer queries [JZW19]. Moreover, the effectiveness of a single substitute model is limited, particularly when targeting adversarially-trained networks or models with significantly different architectures. To address this, later work has explored the use of ensemble surrogates and techniques such as explicit gradient alignment to improve transferability [Gao+19; Che+24]. Another limitation is that the original study only evaluated *untargeted* attacks [Gao+19]. However, *targeted* attacks are generally harder to transfer [Liu+16] and are often considered less effective in practice [CW17].

4.7 Decision-based attacks

Decision-based attacks are black-box attacks, so the adversary only has access to the final class label, called *decision*, of the model. This excludes any internal model information, like confidence scores or output probability. Due to that, decision-based attacks do not use gradients. Instead, these attacks consist of an iterative search for minimal changes that flip the decision of the model. Typically, the method will start with a misclassified input and aim to minimise the modifications needed from the correctly classified input, while keeping the correctly classified label [BRB18b].

In 2018, Brendel et al. proposed **Boundary Attack**, which is one of the most prominent decision-based attacks. The Boundary Attack starts from the current adversarial input $\tilde{o}_{k-1}$, which is already misclassified. After that, it repeats two steps [BRB18b]:

1. It applies a random orthogonal step, exploring new directions around $\tilde{o}_{k-1}$ without moving closer to the original input o . This random step is sampled from a proposal distribution $P(\tilde{o}_{k-1})$.
2. The algorithm takes a small step towards the original input o , which reduces the distance $\|o - \tilde{o}_k\|_2$.

Both steps together form the perturbation η_k . The attack only accepts the new candidate $\tilde{o}_{k-1} + \eta_k$ if it stays misclassified. Otherwise, it lets the previous input unchanged. The step sizes for both components are dynamically adjusted to maintain a balance between exploration (1.) and modification reduction (2.) [BRB18b].

Because the boundary attack only requires access to the decision of the model, it is robust to defences which break gradient-based or score-based attacks, such as gradient masking or stochastic outputs [BRB18b]. Also, unlike transfer-based attacks, boundary attack does not rely on substitute models or training data. The attack scales to complex datasets like ImageNet and produces modifications that are competitive with white-box gradient attacks. Another advantage is its minimal need for hyperparameter tuning, which simplifies its application across different models and datasets [BRB18b].

However, the Boundary Attack is not very efficient. It needs many queries to converge because it cannot use gradient information [BRB18b]. This makes it computationally expensive, especially on high-dimensional data. Additionally, it needs an initial adversarial input, which is sometimes found through inefficient random sampling [BRB18b].

To improve these efficiency problems, Chen et al. proposed **HopSkipJump Attack (HSJA)** [CJW20]. HSJA initially starts with an adversarial input that is already misclassified. Then, the following three steps are repeated [CJW20]:

1. It performs a *binary search* to find a point close to the decision boundary. This makes sure that the following steps work near the boundary, where it is easiest to estimate the correct direction to move.

2. It estimates the gradient direction, which is the direction in which the input should be changed to reduce the perturbations, while staying misclassified. This is done by adding small random changes in different directions and observing whether the decision of the model changes. These observations are combined using a *Monte Carlo estimate*, which means averaging over many random samples to get an approximate direction.
3. The algorithm takes a step along the estimated gradient direction, moving towards the original input and thereby reducing the perturbations. The step size starts large and is then reduced by half until the new candidate still stays misclassified.

Similar to Boundary Attack, the step sizes and the number of random samples for estimating the gradient are dynamically adjusted. [CJW20].

HSJA improves query efficiency by replacing random search with binary search and by estimating the gradient direction at the decision boundary. This allows the attack to operate more efficiently on high-dimensional data. Additionally, HSJA also works for multiple distance metrics and achieves smaller modifications on some tasks, such as ImageNet and CIFAR-10 image classification tasks [CJW20].

However, it still requires many queries compared to white-box attacks. Also, a large number of random samples are required to successfully estimate the gradient direction. On some models, especially those with complex decision boundaries, the attack can still struggle to find minimal perturbations [CJW20]. Despite these disadvantages, HSJA is considered state-of-the-art in decision-based attacks because of its improved efficiency and effectiveness [SSK21].

Decision-based attacks are important because they work on real-world models, where gradients and confidence scores are inaccessible. This includes applications such as autonomous cars or commercial image classification APIs [BRB18b; CJW20].

However, a key limitation of decision-based attacks is their high number of queries, which makes them slow and computationally costly in practical scenarios. Even HSJA requires thousands of queries for targeted attacks on complex datasets like ImageNet [CJW20], despite the improvements it has made. Additionally, both boundary attack and HSJA have primarily been evaluated on image classification tasks such as MNIST, CIFAR, and ImageNet [BRB18b; CJW20]. Therefore, their transferability to other domains remains largely unexplored and has been critically discussed as a limitation in the broader field of decision-based attacks [BRB18b; CJW20].

4.8 ZOO – Zeroth-Order Optimisation Attack

The ZOO attack proposed by Chen et al. [Che+17] operates in a *black-box* setting: the attacker can query the target model and observe only the output scores (confidence scores or probabilities), but has no access to internal gradients, weights, or architectural details. This contrasts with the transfer-based black-box attack discussed earlier – the substitute-model strategy of Papernot et al. [Pap+17] – which learns a surrogate using only the predicted class labels and therefore ignores confidence information entirely.

The ZOO attack builds on the CW- ℓ_2 attack. The key differences are the modified objective function and the black-box setting – it is not possible to calculate gradients directly. Consequently, the authors propose estimating gradients using *zeroth-order optimisation with coordinate descent*. Compared with standard optimisation methods that update all pixels simultaneously, this variant updates only a small batch of pixels at each step, improving computational efficiency.

Remark: The authors use the term *coordinates* to refer to dimensions of the input space – pixels or features; we adopt the same terminology as well.

Objective function. ZOO drives the input toward the target class t with a CW-inspired loss that can be computed purely from the model’s output scores. Chen et al. replace logits $Z_i(x')$ in (1) with the logarithm of softmax probabilities:

$$f(x') = \left(\max_{i \neq t} \log p_i(x') - \log p_t(x') \right)^+, \quad (u)^+ := \max(u, -\kappa),$$

where $p_i(x')$ denotes the predicted probability (confidence score) for class i , and $\kappa \geq 0$ is the confidence margin as in (1). This formulation mitigates numerical instabilities arising when one probability dominates, ensuring more stable gradient estimates in the zeroth-order setting.

ZOO-Adam. In ZOO, gradients are estimated by querying the objective function at two nearby points, avoiding direct gradient calculation through back-propagation. Specifically, for each pixel (coordinate) j , the model is queried twice:

$$\frac{\partial f}{\partial x_j} \approx \frac{f(x + he_j) - f(x - he_j)}{2h},$$

where e_j is the unit basis vector and $h > 0$ is a small constant. The resulting gradient estimate $\hat{\nabla}_x f$ is noisy but sufficient for optimisation.

The estimated gradient is then fed directly into the Adam optimiser. A single iteration proceeds as follows:

1. Select a mini-batch of coordinates – pixels/features (all pixels in the simplest case, or a subset chosen via importance sampling).
2. Estimate gradients for each chosen coordinate using the two-point method.
3. Perform an *Adam* update: $w \leftarrow w - \eta \text{Adam}(\hat{\nabla}_x f)$, mapping w back to a valid image via the CW tanh-reparameterisation.

Query-efficiency tricks. Using zeroth-order optimisation to attack large models and high-dimensional input spaces can become prohibitively expensive because of the need to estimate a large number of gradients. To address this, the authors introduce several heuristics:

- a. *Attack-space reduction.* Apply a transformation that reduces the dimensionality of the adversarial noise added to the original image. The noise is optimised in a lower-dimensional space and then up-sampled, significantly reducing the number of coordinate estimations required.
- b. *Hierarchical search.* The attack starts in a very low-dimensional space, gradually increase the dimension of the adversarial noise, and stops when a valid adversarial example is found.
- c. *Importance sampling.* Chen et al. [Che+17] divide the image into 8×8 regions and assign each a sampling probability score based on the maximum pixel change observed so far in that region. At each step, pixels are sampled according to the scores, focusing updates on high-impact areas. The scores are updated periodically to adapt to changes during optimisation.
- d. *State reset.* After the first successful adversarial example is found, reset Adam’s state to reduce the final distortion.

Chen et al. [Che+17] evaluated ZOO on MNIST, CIFAR-10, and ImageNet (Inception-v3). They report:

- *Success rates:* Comparable to the white-box CW- ℓ_2 attack – nearly 100% on MNIST/CIFAR-10 and approximately 90% on ImageNet.
- *Distortion:* Adversarial examples with similarly low ℓ_2 norms and visually imperceptible perturbations.
- *Query cost:* Higher than white-box methods, requiring thousands to millions of model queries, but far fewer than substitute-model black-box attacks.

Overall, ZOO succeeded in attaining performance comparable to white-box attacks and significantly outperformed earlier transfer-based black-box methods [Che+17]. However, a key limitation lies in its relatively high query complexity, which has led to its obsolescence in light of more recent black-box attacks. Methods such as GenAttack, AutoZOOM, and SurFree achieve similar success rates and distortion levels, but require dramatically fewer model queries. These newer approaches significantly improve query efficiency while maintaining high attack effectiveness [Alz+18; Tu+18; MFM20].

Modern improvements beyond ZOO:

- **AutoZOOM** [Tu+18]: Incorporates autoencoders to project optimisation into a lower-dimensional space and uses adaptive gradient estimation, reducing query counts by $>90\%$ while maintaining high attack success and low distortion.
- **GenAttack** [Alz+18]: A gradient-free, genetic algorithm-based method that uses only a few thousand queries (around 2,000 for MNIST/CIFAR-10, over $200\times$ fewer than ZOO for ImageNet) with comparable attack performance.
- **SurFree** [MFM20]: A surrogate-free black-box attack that abandons gradient estimation in favour of boundary-following strategies. It achieves state-of-the-art efficiency under strict query constraints.

While ZOO was groundbreaking in demonstrating that white-box-level adversarial attacks could be performed in a pure black-box setting with only score outputs, it is no longer state-of-the-art. Methods like AutoZOOM, GenAttack, and SurFree provide better trade-offs between success rate, distortion, and query efficiency.

4.9 Poisoning Attacks

Up until now, all attack methods discussed are *evasion attacks*. In these attacks, adversarial examples are presented to an already trained model at inference time.

Now, we introduce *poisoning attacks*, where adversarial examples are injected into the training data of a model. This is done before the model is trained to manipulate the learning process of the model [Gei+21].

In this section, we focus on *targeted poisoning attacks* that operate in a white-box or grey-box setting, where the attacker has at least some knowledge of the model architecture or training process. The objective is to cause the model to misclassify a specific target sample while maintaining high accuracy on the remaining test data [Muñ+17].

Bilevel Optimisation Poisoning. Bilevel Optimisation Poisoning was introduced in [Muñ+17] to provide a theoretical mathematical foundation for poisoning attacks. It formulates the attack as a nested optimisation problem [Muñ+17]: the outer problem optimises the poisoning data to maximise an adversarial objective, while the inner problem represents the standard training procedure of the model:

$$\hat{D}^* \in \arg \max_{\hat{D} \in \mathcal{D}_{\text{poison}}} \mathcal{R}(\theta^*(\hat{D})) \quad \text{such that} \quad \theta^*(\hat{D}) \in \arg \min_{\theta} \mathcal{L}(\theta, D \cup \hat{D})$$

Here, $\mathcal{L}$ is the training loss, D is the clean training data, $\hat{D}$ is the poisoning data and $\mathcal{R}$ is the adversarial objective defined by the attacker.

The main advantage of Bilevel Optimisation Poisoning is its solid theoretical foundation: it computes an optimal perturbation strategy. [Muñ+17]

However, Bilevel optimisation Poisoning is computationally expensive because it requires backpropagating through the entire training loop. This makes it impractical for large models or datasets, and limits its use to white-box scenarios with complete model access. [Muñ+17]

Gradient Matching Poisoning. To address this computational inefficiency, Geiping et al. introduced Gradient Matching Poisoning [Gei+21].

The core idea is to craft poisoned inputs such that their contribution to the gradient closely aligns with the gradient of a target instance [Gei+21]. This approximation avoids full backpropagation through training and makes the attack scalable. The method works in a grey-box setting and performs targeted attacks using clean-label data [Gei+21]. The attacker only perturbs a small subset of training samples, keeping their labels unchanged.

Formally, the attack minimises the cosine distance between the target gradient and the average gradient of the poisoned examples [Gei+21]:

$$\min_{\Delta \in \mathcal{C}} \left(1 - \frac{\left\langle \nabla_{\theta} \mathcal{L}(x_t, \theta), \sum_{i=1}^P \nabla_{\theta} \mathcal{L}(x_i + \Delta_i, \theta) \right\rangle}{\left\| \nabla_{\theta} \mathcal{L}(x_t, \theta) \right\| \cdot \left\| \sum_{i=1}^P \nabla_{\theta} \mathcal{L}(x_i + \Delta_i, \theta) \right\|} \right)$$

where x_t is the target sample, x_i are the poisoned inputs and Δ_i are the bounded perturbations constrained by $\mathcal{C} = \{\Delta : \|\Delta_i\|_{\infty} \leq \epsilon\}$.

Compared to bilevel optimisation, Gradient Matching is significantly more efficient. It operates on a single pretrained model, uses only a subset of the training data, and achieves successful targeted poisoning with a perturbation budget as low as 0.1% of the dataset [Gei+21].

The attack is hidden, since the poisoned samples keep their correct semantic labels and remain visually similar to benign inputs. However, a key limitation is that it requires some knowledge of the architecture of the victim model and does not generalise to untargeted poisoning [Gei+21].

Gradient Matching currently represents the state-of-the-art in poisoning attacks due to its unique combination of computational efficiency, effectiveness on large-scale datasets like ImageNet and strong ability to stay hidden. It is the first method to successfully poison deep neural networks trained from scratch using only imperceptibly perturbed, clean-label data [Gei+21].

Poisoning attacks pose a critical security threat to machine learning systems, especially when training data is sourced from untrusted origins. Bilevel optimisation [Muñ+17] provides a theoretically optimal formulation but is computationally infeasible for modern networks. Gradient Matching Poisoning [Gei+21] builds upon this foundation and introduces a scalable, practical attack that achieves targeted poisoning with minimal training data manipulation. Its success on large-scale models highlights the urgent need for robust data pipelines and pre-training data verification mechanisms to prevent silent model corruption.

5 Conclusion

In this paper, we analysed the progression of adversarial attacks in the image domain. These attacks included evasion attacks and poisoning attacks.

We structured the evaluation of the different attack methods around the attack goal, the level of model knowledge and the mathematical constraints imposed by threat norms. By following this categorisation, we demonstrated how white-box attacks have evolved from simple gradient-based methods like FGSM or PGD to AutoAttack, a complex parameter-free approach that exploits specific model weaknesses.

Additionally, the different types of black-box attacks presented demonstrate that adversarial attacks do not depend on internal model knowledge, as the block-box attacks presented are sufficiently effective.

Furthermore, despite their high computational cost, poisoning attacks demonstrate that model vulnerabilities are not limited to perturbations at inference time. The timing of these attacks during training emphasises the importance of trustworthy data sources.

Despite the significant progress that has been made over the last ten years in understanding and evaluating adversarial robustness, many open questions remain.

First, most research has been done on adversarial attacks in the image domain, while other application areas such as audio processing, malware classification or spam detection have not been researched equally. Therefore, these areas should be the focus of further investigations.

Second, many existing adversarial defences only address specific attack methods or settings and do not provide general robustness guarantees across different attack types and domains. Future work should focus on developing practical defences that protect against a wide range of adversarial attacks and domains.

In general, the study of adversarial attacks remains important to improve the reliability and safety of machine learning systems in real-world applications.

6 Statement of Contribution

The contributions are divided as follows:

Line-based Contribution (Exact Editing Responsibility)

- **Lara:** Lines 19-41, 159-222, 270-299, 393-494, 534-600, 676-754
- **Alina:** Lines 7-12, 42-158, 223-269, 300-304, 310-392, 495-533, 601-675
- **Joint Contribution (Lara and Alina):** Lines 13-18, 305-309

Section-based Contribution (Main Thematic Contribution)

- **Lara:** Introduction, Adversarial Attacks, PGD, AutoAttack, Decision-Based Attacks, Poisoning Attacks, Conclusion
- **Alina:** Overview of Basic Concepts, Adversarial Examples, FGSM, BIM, CW Attacks, Substitute Models, ZOO

References

- [CW17] Nicholas Carlini and David Wagner. “Towards evaluating the robustness of neural networks”. In: *IEEE Symposium on Security and Privacy* (2017).
- [Mad+18] Aleksander Madry et al. “Towards deep learning models resistant to adversarial attacks”. In: *International Conference on Learning Representations (ICLR)*. Originally posted as arXiv preprint arXiv:1706.06083. 2018. URL: <https://arxiv.org/abs/1706.06083>.
- [Sze+14] Christian Szegedy et al. *Intriguing properties of neural networks*. 2014. arXiv: 1312.6199 [cs.CV]. URL: <https://arxiv.org/abs/1312.6199>.
- [GSS15] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. “Explaining and Harnessing Adversarial Examples”. In: *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*. Ed. by Yoshua Bengio and Yann LeCun. 2015. URL: <http://arxiv.org/abs/1412.6572>.
- [CW16] Nicholas Carlini and David Wagner. “Hidden Voice Commands”. In: *Proceedings of the 25th USENIX Security Symposium*. USENIX Association, 2016, pp. 513–530. URL: <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/carlini>.
- [BR18] Battista Biggio and Fabio Roli. “Wild patterns: Ten years after the rise of adversarial machine learning”. In: *Pattern Recognition* 84 (2018), pp. 317–331.
- [PY20] Daniel Park and Bülent Yener. *A survey on practical adversarial examples for malware classifiers*. Tech. rep. arXiv, 2020. URL: <https://arxiv.org/abs/2011.05973>.
- [Sch+18] Ludwig Schmidt et al. “Adversarially Robust Generalization Requires More Data”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2018.
- [Den12] Li Deng. “The mnist database of handwritten digit images for machine learning research”. In: *IEEE Signal Processing Magazine* 29.6 (2012), pp. 141–142.
- [Goo24] Google Developers. *Machine Learning Glossary: Logits*. Accessed: 2025-07-01. 2024. URL: <https://developers.google.com/machine-learning/glossary/#logits>.
- [GBC16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [Ter+25] Juan Terven et al. “A comprehensive survey of loss functions and metrics in deep learning”. In: *Artificial Intelligence Review* 58.7 (Apr. 2025). ISSN: 1573-7462. DOI: 10.1007/s10462-025-11198-7. URL: <http://dx.doi.org/10.1007/s10462-025-11198-7>.
- [Rud16] Sebastian Ruder. “An overview of gradient descent optimization algorithms”. In: *CoRR* abs/1609.04747 (2016). arXiv: 1609.04747. URL: <http://arxiv.org/abs/1609.04747>.
- [KSH12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems 25: 26th Annual Conference on Neural Information Processing Systems 2012. Proceedings of a meeting held December 3-6, 2012, Lake Tahoe, Nevada, United States*. Ed. by Peter L. Bartlett et al. 2012, pp. 1106–1114. URL: <https://proceedings.neurips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html>.

- [ZL20] Jiliang Zhang and Chen Li. “Adversarial Examples: Opportunities and Challenges”. In: *IEEE Trans. Neural Networks Learn. Syst.* 31.7 (2020), pp. 2578–2593. DOI: [10.1109/TNNLS.2019.2933524](https://doi.org/10.1109/TNNLS.2019.2933524). URL: <https://doi.org/10.1109/TNNLS.2019.2933524>.
- [GSS14] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. “Explaining and harnessing adversarial examples”. In: *arXiv preprint arXiv:1412.6572* (2014).
- [PMG17] Nicolas Papernot, Patrick McDaniel, and Ian Goodfellow. “Practical black-box attacks against machine learning”. In: *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security* (2017).
- [BRB18a] Wieland Brendel, Jonas Rauber, and Matthias Bethge. “Decision-based adversarial attacks: Reliable attacks against black-box machine learning models”. In: *International Conference on Learning Representations (ICLR)* (2018).
- [Gei+24] Simon Geisler et al. *Attacking Large Language Models with Projected Gradient Descent*. Preprint. 2024. URL: <https://arxiv.org/abs/2402.09154>.
- [Els+18] Gamaleldin F. Elsayed et al. “Adversarial Examples that Fool both Computer Vision and Time-Limited Humans”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2018. URL: <https://arxiv.org/abs/1802.08195>.
- [Eyk+18] Kevin Eykholt et al. “Robust Physical-World Attacks on Deep Learning Visual Classification”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 1625–1634.
- [Yua+17] Xiaoyong Yuan et al. “Adversarial Examples: Attacks and Defenses for Deep Learning”. In: *CoRR* abs/1712.07107 (2017). arXiv: [1712.07107](https://arxiv.org/abs/1712.07107). URL: <http://arxiv.org/abs/1712.07107>.
- [Akh+21] Naveed Akhtar et al. “Threat of Adversarial Attacks on Deep Learning in Computer Vision: Survey II”. In: *CoRR* abs/2108.00401 (2021). arXiv: [2108.00401](https://arxiv.org/abs/2108.00401). URL: <https://arxiv.org/abs/2108.00401>.
- [KGB16] Alexey Kurakin, Ian J. Goodfellow, and Samy Bengio. “Adversarial examples in the physical world”. In: *CoRR* abs/1607.02533 (2016). arXiv: [1607.02533](https://arxiv.org/abs/1607.02533). URL: <http://arxiv.org/abs/1607.02533>.
- [Den+09] Jia Deng et al. “ImageNet: A large-scale hierarchical image database”. In: *2009 IEEE Conference on Computer Vision and Pattern Recognition*. 2009, pp. 248–255. DOI: [10.1109/CVPR.2009.5206848](https://doi.org/10.1109/CVPR.2009.5206848).
- [KNH+14] Alex Krizhevsky, Vinod Nair, Geoffrey Hinton, et al. “The CIFAR-10 dataset”. In: *online: http://www.cs.toronto.edu/kriz/cifar.html* 55.5 (2014), p. 2.
- [CH20] Francesco Croce and Matthias Hein. “Reliable Evaluation of Adversarial Robustness with an Ensemble of Diverse Parameter-Free Attacks”. In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*. introduced AutoAttack: APGD-CE, APGD-DLR, FAB, Square Attack. 2020, pp. 2206–2216.
- [SBR18] Mahmood Sharif, Lujo Bauer, and Michael K. Reiter. “On the Suitability of Lp-Norms for Creating and Preventing Adversarial Examples”. In: *2018 IEEE Conference on Computer Vision and Pattern Recognition Workshops, CVPR Workshops 2018, Salt Lake City, UT, USA, June 18-22, 2018*. Computer Vision Foundation / IEEE Computer Society, 2018, pp. 1605–1613. DOI: [10.1109/CVPRW.2018.00211](https://doi.org/10.1109/CVPRW.2018.00211). URL: http://openaccess.thecvf.com/content%5C_cvpr%5C_2018%5C_workshops/w32/html/Sharif%5C_On%5C_the%5C_Suitability%5C_CVPR%5C_2018%5C_paper.html.
- [Man+23] Ravi Mangal et al. “Is Certifying ℓ_{pp} Robustness Still Worthwhile?” In: *CoRR* abs/2310.09361 (2023). DOI: [10.48550/ARXIV.2310.09361](https://doi.org/10.48550/ARXIV.2310.09361). arXiv: [2310.09361](https://arxiv.org/abs/2310.09361). URL: <https://doi.org/10.48550/arXiv.2310.09361>.
- [Pap+16] Nicolas Papernot et al. “Distillation as a Defense to Adversarial Perturbations Against Deep Neural Networks”. In: *IEEE Symposium on Security and Privacy, SP 2016, San Jose, CA, USA, May 22-26, 2016*. IEEE Computer Society, 2016, pp. 582–597. DOI: [10.1109/SP.2016.41](https://doi.org/10.1109/SP.2016.41). URL: <https://doi.org/10.1109/SP.2016.41>.
- [Che+17] Pin-Yu Chen et al. “ZOO: Zeroth Order Optimization Based Black-box Attacks to Deep Neural Networks without Training Substitute Models”. In: *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security, AISec@CCS 2017, Dallas, TX,*

- USA, November 3, 2017. Ed. by Bhavani Thuraisingham et al. ACM, 2017, pp. 15–26. DOI: [10.1145/3128572.3140448](https://doi.org/10.1145/3128572.3140448). URL: <https://doi.org/10.1145/3128572.3140448>.
- [And+20] Maksym Andriushchenko et al. “Square Attack: a query-efficient black-box adversarial attack via random search”. In: *European Conference on Computer Vision (ECCV)*. 2020. DOI: [10.1007/978-3-030-58592-1_29](https://doi.org/10.1007/978-3-030-58592-1_29).
- [Lor+22] Peter Lorenz et al. “Is RobustBench/AutoAttack a Suitable Benchmark for Adversarial Robustness?” In: *AAAI-22 Workshop on Adversarial Machine Learning and Beyond*. critiques RobustBench’s use of AutoAttack on CIFAR-10. 2022.
- [Pap+17] Nicolas Papernot et al. “Practical Black-Box Attacks against Machine Learning”. In: *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security, AsiaCCS 2017, Abu Dhabi, United Arab Emirates, April 2-6, 2017*. Ed. by Ramesh Karri et al. ACM, 2017, pp. 506–519. DOI: [10.1145/3052973.3053009](https://doi.org/10.1145/3052973.3053009). URL: <https://doi.org/10.1145/3052973.3053009>.
- [Car+19] Nicholas Carlini et al. “On Evaluating Adversarial Robustness”. In: *CoRR abs/1902.06705* (2019). arXiv: [1902.06705](https://arxiv.org/abs/1902.06705). URL: <http://arxiv.org/abs/1902.06705>.
- [JZW19] Xue Jiang, Xiao Zhang, and Dongrui Wu. “Active Learning for Black-Box Adversarial Attacks in EEG-Based Brain-Computer Interfaces”. In: *CoRR abs/1911.04338* (2019). arXiv: [1911.04338](https://arxiv.org/abs/1911.04338). URL: <http://arxiv.org/abs/1911.04338>.
- [Gao+19] Xianfeng Gao et al. “Boosting Targeted Black-Box Attacks via Ensemble Substitute Training and Linear Augmentation”. In: *Applied Sciences* 9.11 (2019). ISSN: 2076-3417. DOI: [10.3390/app9112286](https://doi.org/10.3390/app9112286). URL: <https://www.mdpi.com/2076-3417/9/11/2286>.
- [Che+24] Huanran Chen et al. “Rethinking Model Ensemble in Transfer-based Adversarial Attacks”. In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=AcJrSoArlh>.
- [Liu+16] Yanpei Liu et al. “Delving into Transferable Adversarial Examples and Black-box Attacks”. In: *CoRR abs/1611.02770* (2016). arXiv: [1611.02770](https://arxiv.org/abs/1611.02770). URL: <http://arxiv.org/abs/1611.02770>.
- [BRB18b] Wieland Brendel, Jonas Rauber, and Matthias Bethge. “Decision-Based Adversarial Attacks: Reliable Attacks Against Black-Box Machine Learning Models”. In: *International Conference on Learning Representations (ICLR)*. 2018. URL: <https://arxiv.org/abs/1712.04248>.
- [CJW20] Jianbo Chen, Michael I. Jordan, and Martin J. Wainwright. “HopSkipJumpAttack: A Query-Efficient Decision-Based Attack”. In: *Proceedings of the 2020 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2020, pp. 1277–1294. URL: <https://arxiv.org/abs/1904.02144>.
- [SSK21] Carl-Johann Simon-Gabriel, Noman Ahmed Sheikh, and Andreas Krause. “PopSkipJump: Decision-Based Attack for Probabilistic Classifiers”. In: *Proceedings of the 38th International Conference on Machine Learning (ICML)*. Vol. 139. Proceedings of Machine Learning Research. PMLR, 2021, pp. 9253–9263. URL: <https://proceedings.mlr.press/v139/simon-gabriel21a.html>.
- [Alz+18] Moustafa Alzantot et al. “GenAttack: Practical Black-box Attacks with Gradient-Free Optimization”. In: *CoRR abs/1805.11090* (2018). arXiv: [1805.11090](https://arxiv.org/abs/1805.11090). URL: <http://arxiv.org/abs/1805.11090>.
- [Tu+18] Chun-Chen Tu et al. “AutoZOOM: Autoencoder-based Zeroth Order Optimization Method for Attacking Black-box Neural Networks”. In: *CoRR abs/1805.11770* (2018). arXiv: [1805.11770](https://arxiv.org/abs/1805.11770). URL: <http://arxiv.org/abs/1805.11770>.
- [MFM20] Thibault Maho, Teddy Furon, and Erwan Le Merrer. “SurFree: a fast surrogate-free black-box attack”. In: *CoRR abs/2011.12807* (2020). arXiv: [2011.12807](https://arxiv.org/abs/2011.12807). URL: <https://arxiv.org/abs/2011.12807>.
- [Gei+21] Jonas Geiping et al. “Witches’ brew: Industrial scale data poisoning via gradient matching”. In: *International Conference on Learning Representations (ICLR)*. 2021. URL: <https://openreview.net/forum?id=ZR7eYOyQuf>.

- 917 [Muñ+17] Luis Muñoz-González et al. “Towards poisoning of deep learning algorithms with
918 back-gradient optimization”. In: *Proceedings of the 10th ACM Workshop on Artificial*
919 *Intelligence and Security*. ACM. 2017, pp. 27–38. DOI: [10.1145/3128572.3140451](https://doi.org/10.1145/3128572.3140451).

Zentrales Prüfungsamt/Central Examination Office



Eidesstattliche Versicherung

Statutory Declaration in Lieu of an Oath

Shcherbakova Alina
Name, Vorname/Last Name, First Name

463616
Matrikelnummer (freiwillige Angabe)
Matriculation No. (optional)

Ich versichere hiermit an Eides Statt, dass ich die vorliegende Arbeit/Bachelorarbeit/
Masterarbeit* mit dem Titel

I hereby declare in lieu of an oath that I have completed the present paper/Bachelor thesis/Master thesis* entitled

Adversarial attacks (AA)

selbstständig und ohne unzulässige fremde Hilfe (insbes. akademisches Ghostwriting)
erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt.
Für den Fall, dass die Arbeit zusätzlich auf einem Datenträger eingereicht wird, erkläre ich,
dass die schriftliche und die elektronische Form vollständig übereinstimmen. Die Arbeit hat in
gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

independently and without illegitimate assistance from third parties (such as academic ghostwriters). I have used no other than
the specified sources and aids. In case that the thesis is additionally submitted in an electronic format, I declare that the written
and electronic versions are fully identical. The thesis has not been submitted to any examination body in this, or similar, form.

Aachen 27.07.2025
Ort, Datum/City, Date

[Signature]
Unterschrift/Signature

*Nichtzutreffendes bitte streichen

*Please delete as appropriate

Belehrung:

Official Notification:

§ 156 StGB: Falsche Versicherung an Eides Statt

Wer vor einer zur Abnahme einer Versicherung an Eides Statt zuständigen Behörde eine solche Versicherung
falsch abgibt oder unter Berufung auf eine solche Versicherung falsch aussagt, wird mit Freiheitsstrafe bis zu drei
Jahren oder mit Geldstrafe bestraft.

Para. 156 StGB (German Criminal Code): False Statutory Declarations

Whoever before a public authority competent to administer statutory declarations falsely makes such a declaration or falsely
testifies while referring to such a declaration shall be liable to imprisonment not exceeding three years or a fine.

§ 161 StGB: Fahrlässiger Falscheid; fahrlässige falsche Versicherung an Eides Statt

(1) Wenn eine der in den §§ 154 bis 156 bezeichneten Handlungen aus Fahrlässigkeit begangen worden ist, so
tritt Freiheitsstrafe bis zu einem Jahr oder Geldstrafe ein.

(2) Strafflosigkeit tritt ein, wenn der Täter die falsche Angabe rechtzeitig berichtet. Die Vorschriften des § 158
Abs. 2 und 3 gelten entsprechend.

Para. 161 StGB (German Criminal Code): False Statutory Declarations Due to Negligence

(1) If a person commits one of the offences listed in sections 154 through 156 negligently the penalty shall be imprisonment not
exceeding one year or a fine.

(2) The offender shall be exempt from liability if he or she corrects their false testimony in time. The provisions of section 158 (2)
and (3) shall apply accordingly.

Die vorstehende Belehrung habe ich zur Kenntnis genommen:

I have read and understood the above official notification:

Aachen 27.07.2025
Ort, Datum/City, Date

[Signature]
Unterschrift/Signature

Adversarial Attacks (AA) 21

891

Zentrales Prüfungsamt/Central Examination Office

RWTH AACHEN
UNIVERSITY

Eidesstattliche Versicherung Statutory Declaration in Lieu of an Oath

Freubmann, Lara 457008
Name, Vorname/Last Name, First Name Matrikelnummer (freiwillige Angabe)
Matriculation No. (optional)

Ich versichere hiermit an Eides Statt, dass ich die vorliegende Arbeit/Bachelorarbeit/
Masterarbeit* mit dem Titel
I hereby declare in lieu of an oath that I have completed the present paper/Bachelor thesis/Master thesis* entitled
Adversarial Attacks (AA)

selbstständig und ohne unzulässige fremde Hilfe (insbes. akademisches Ghostwriting)
erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt.
Für den Fall, dass die Arbeit zusätzlich auf einem Datenträger eingereicht wird, erkläre ich,
dass die schriftliche und die elektronische Form vollständig übereinstimmen. Die Arbeit hat in
gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.
Independently and without illegitimate assistance from third parties (such as academic ghostwriters) I have used no other than
the specified sources and aids. In case that the thesis is additionally submitted in an electronic format, I declare that the written
and electronic versions are fully identical. The thesis has not been submitted to any examination body in this, or similar, form.

Sankt Augustin, 27.07.25 Freubmann
Ort, Datum/city, Date Unterschrift/Signature
*Nichtzutreffendes bitte streichen.
*Please delete as appropriate

Belehrung:
Official Notification:

§ 156 StGB: Falsche Versicherung an Eides Statt
Wer vor einer zur Abnahme einer Versicherung an Eides Statt zuständigen Behörde eine solche Versicherung
falsch abgibt oder unter Berufung auf eine solche Versicherung falsch aussagt, wird mit Freiheitsstrafe bis zu drei
Jahren oder mit Geldstrafe bestraft.
Para. 156 StGB (German Criminal Code): False Statutory Declarations
Whoever before a public authority competent to administer statutory declarations falsely makes such a declaration or falsely
testifies while referring to such a declaration shall be liable to imprisonment not exceeding three years or a fine

§ 161 StGB: Fahrlässige Falschheit; fahrlässige falsche Versicherung an Eides Statt
(1) Wenn eine der in den §§ 154 bis 156 bezeichneten Handlungen aus Fahrlässigkeit begangen worden ist, so
trifft Freiheitsstrafe bis zu einem Jahr oder Geldstrafe ein.
(2) Strafflosigkeit tritt ein, wenn der Täter die falsche Angabe rechtzeitig berichtigt. Die Vorschriften des § 158
Abs. 2 und 3 gelten entsprechend.
Para. 161 StGB (German Criminal Code): False Statutory Declarations Due to Negligence
(1) If a person commits one of the offences listed in sections 154 through 156 negligently the penalty shall be imprisonment not
exceeding one year or a fine
(2) The offender shall be exempt from liability if he or she corrects their false testimony in time. The provisions of section 158 (2)
and (3) shall apply accordingly.

Die vorstehende Belehrung habe ich zur Kenntnis genommen.
I have read and understood the above official notification.

Sankt Augustin, 27.07.25 Freubmann
Ort, Datum/City, Date Unterschrift/Signature